Symphonym: Universal Phonetic Embeddings for Cross-Script Toponym Matching via Teacher-Student Distillation

Stephen Gadd
ORCiD: 0000-0003-3060-0181
World Historical Gazetteer / University of Pittsburgh
stephen@docuracy.co.uk

Draft: January 10, 2026

Abstract

Linking place names across historical sources, languages, and writing systems remains a fundamental challenge in digital humanities and geographic information retrieval. Existing approaches either require language-specific phonetic algorithms, expert-curated transliteration rules, or fail to capture the phonetic relationships that make "Moscow", "Москва", and "موسكو" recognisably the same place. I present **Symphonym**, a neural embedding system that maps toponyms from any script into a unified 128-dimensional phonetic space, enabling direct similarity comparison without runtime phonetic conversion or language pair-specific resources. Symphonym uses a Teacher-Student architecture: a Teacher network trained on articulatory phonetic features (derived via Epitran grapheme-to-phoneme conversion and PanPhon feature extraction) produces target embeddings, while a Student network learns to approximate these embeddings directly from characters, guided by script and optional language embeddings. The Student operates at inference time without requiring phonetic resources, enabling deployment at scale. I train on co-located toponym pairs from GeoNames, Wikidata, and TGN, applying phonetic similarity filtering to exclude false cognates. A three-phase curriculum progresses from phonetic feature learning through knowledge distillation to hard negative discrimination. [Results pending: cross-script retrieval accuracy, noise robustness.]

1 Introduction

Historical gazetteers aggregate place references from sources spanning millennia, dozens of languages, and multiple writing systems. The World Historical Gazetteer[1] (WHG) alone indexes over 112 million toponym records from antiquity to the present, drawn from sources as diverse as classical Latin itineraries,

[1] https://whgazetteer.org

medieval Arabic geographies, and modern national mapping agencies. Each toponym record is tagged with language and script metadata, though the same surface form (e.g., "London") may appear multiple times when attested in different languages. Linking these references—determining that "Londinium", "Londres", "Лондон", and "伦敦" all denote the same city—is essential for cross-cultural historical research but remains technically challenging.

The core difficulty is that place name similarity is fundamentally *phonetic*, not orthographic. English speakers recognise "Munich" and "München" as the same city not because the spellings match, but because the sounds are similar enough to suggest common origin. Yet computational approaches to toponym matching have largely relied on string-based metrics (edit distance, Jaro-Winkler) or phonetic algorithms designed for single languages (Soundex, Metaphone for English; Cologne phonetic for German). These approaches fail catastrophically when names cross script boundaries: no amount of edit distance tuning will link "東京" to "Tokyo".

Recent work has applied neural embeddings to geographic entity matching, but existing systems either operate within single scripts, require language identification at inference time, or depend on curated transliteration resources that don't scale to the long tail of historical languages. Until now, there has been no system that can place "Москва", "Moscow", "موسكو", and "モスクワ" (Japanese Katakana) near each other in embedding space using only the raw character sequences as input.

This paper presents such a system, which I call *Symphonym*. The key contributions are:

1. A **script-aware but script-agnostic embedding architecture** that maps toponyms from 20+ writing systems into a unified 128-dimensional space where proximity reflects phonetic similarity.

2. A **Teacher-Student training framework** that transfers phonetic knowledge from articulatory features (available only for IPA-transcribed names) to a character-level model requiring no phonetic resources at inference time.

3. A **noise-aware training regime** with strategic language dropout that forces the model to learn script-intrinsic phonetic patterns rather than relying on language-specific shortcuts.

4. **Phonetic similarity filtering** for training data that prevents the model from learning false equivalences between unrelated exonyms (e.g., "Germany" $\neq$ "Deutschland").

5. A **three-phase curriculum** progressing from phonetic feature learning through knowledge distillation to hard negative discrimination.

The resulting system enables fuzzy phonetic search across the full WHG corpus without language identification, script-specific preprocessing, or runtime phonetic conversion.

Scope and Integration. This work addresses the *name-matching* component of toponym resolution specifically. The model has no access to geographic coordinates or spatial context—it operates purely on phonetic similarity between strings. Both Symphonym and WHG's reconciliation pipeline (termed WHG PLACE: Place Linkage, Alignment, and Concordance Engine) build on architecture developed by Grossner and Mostern (2021). Within PLACE, phonetic similarity serves as one input to a larger process: candidate toponyms are first retrieved via the phonetic-aware toponym index, then places containing matching toponyms are selected from the places index and filtered by geographic proximity and other contextual criteria. The contribution here is the phonetic similarity component alone.

In practical terms, Symphonym enhances WHG's Reconciliation Service API[2] (v2.0), facilitating the work of digital humanities and GLAM (galleries, libraries, archives, museums) professionals who seek to link place references in historical documents, archival catalogues, and research datasets to WHG's consolidated index. Once linked, these references become entry points to a web of related materials—other sources attesting the same places, variant historical names, and cross-references across collections—even when the original materials span multiple scripts, languages, and historical periods.

Remaining Work

1. ~~Phase 1 training~~ *Complete (50 epochs, best val_loss=0.0040)*
2. ~~Phase 2 training~~ *Complete (5 epochs, best val_loss=0.0342, val_sim=0.9663)*
3. ~~Phase 3 training~~ *In progress (Epoch 10/30, 33% complete; 97.1% test pass rate, 29/29 cross-script)*
4. ~~MEHDIE benchmark evaluation~~ *Complete (R@1=0.892, MRR=0.930, outperforms baselines)*
5. Generate training curves figure
6. Complete remaining evaluation tasks (cross-script retrieval, noise robustness, historical variants)

[2] https://docs.whgazetteer.org/content/technical/apis.html#reconciliation-service-api

Sections Still Requiring Updates After Retraining

These sections have
placeholder markers and need actual data:

1. Abstract: Results pending placeholder
2. Section 6.4 (Results): Placeholder
3. Section 7 (Discussion): Placeholder
4. Section 8 (Conclusion): Summary placeholder
5. Data/Code Availability: URLs are placeholders

Recommendations for Future Edits

1. Add training curves: Plot loss curves for each phase when available - use phonetics.training.plot_curves
2. Add confusion matrix: For script-pair performance analysis
3. Consider appendix: For detailed per-script evaluation tables
4. Add hyperparameter sensitivity: Brief analysis of key hyperparameters

2 Related Work

2.1 Classical Phonetic Algorithms

The earliest computational approaches to phonetic matching emerged from English-language record linkage. Soundex (Russell, 1918), developed for US Census indexing, maps names to four-character codes by retaining the first letter and encoding subsequent consonants. Its English-centricity is explicit: the algorithm assumes Latin script and English phoneme-grapheme correspondences. Metaphone (Philips, 1990) and Double Metaphone (Philips, 2000) improve accuracy for English by handling common spelling variations, but remain fundamentally tied to English phonology.

Language-specific variants exist for other European languages, but each requires separate implementation and fails outside its target language family. No classical algorithm handles cross-script matching: they assume a single alphabet and a known phoneme inventory.

2.2 String Similarity Metrics

Edit distance metrics (Levenshtein, Damerau-Levenshtein, Jaro-Winkler) measure orthographic rather than phonetic similarity. While useful for detecting OCR errors or spelling variants within a script, they cannot capture phonetic relationships across scripts. The Levenshtein distance between "Moscow" and "Москва" is undefined without transliteration; even after romanisation to "Moskva", the distance of 2 fails to reflect the near-identical pronunciation.

Recchia and Louwerse (Recchia and Louwerse, 2013) evaluated 21 similarity measures on romanised toponyms from 11 countries, finding that optimal measures varied by language but were consistent within linguistic families. Santos et al. (Santos et al., 2018) demonstrated that combining multiple string metrics with machine learning classifiers improved matching accuracy for Portuguese toponyms.

Phonetic-aware edit distances (Kondrak, 2000) weight substitutions by articulatory similarity, but still require input in a common alphabet.

2.3 Neural Approaches to Entity Matching

Recent work applies neural embeddings to geographic entity resolution. Qiu et al. (Qiu et al., 2024) proposed GeoBERT, combining a BERT model pre-trained on Chinese geographic text with the Enhanced Sequential Inference Model (ESIM) architecture for toponym pair classification. Their approach integrates multiple Chinese-specific features including Pinyin romanisation, Wubi keyboard encoding, character radicals, and stroke counts. On Chinese address matching datasets, GeoBERT achieved F1 scores exceeding 0.97. However, the Chinese-specific features have no analogues in other writing systems, and the geographic pre-training corpus is monolingual, limiting cross-lingual application.

Most relevant is the MEHDIE project (Sagi et al., 2025), which developed tools for matching toponyms between historical Hebrew and Arabic sources. Sagi et al. present a benchmark comprising place pairs from 13th-century Arabic geographical texts and 12th-century Hebrew travel narratives. Their approach investigates direct transliteration between Hebrew and Arabic scripts using Phonetisaurus (Novak et al., 2016) for grapheme-to-phoneme conversion and a custom rule-based transliteration library (Translit) that exploits the phonetic affinity between Semitic languages.

The MEHDIE work demonstrates that direct transliteration between related scripts often outperforms romanisation, which loses phonetic information in standard Latin schemes. However, their approach requires language-pair-specific rules: the Hebrew-Arabic transliteration exploits shared Semitic phonology that does not generalise to unrelated language pairs. In contrast, my use of Epitran for G2P conversion supports over 100 languages with a unified IPA output, and PanPhon provides language-agnostic articulatory features. This enables my model to learn phonetic relationships across arbitrary script pairs without requiring man-

ual transliteration rules for each combination.

Rama (Rama, 2016) applied Siamese convolutional networks to cognate identification using character embeddings, demonstrating that neural architectures can learn sound correspondences between related languages. The key distinction from my work lies in the *phonetic priors* provided to the model:

- **Rama (Implicit Learning):** The Siamese CNN operates on raw character embeddings, requiring the model to learn from scratch that characters like 'b' and 'p' are phonetically related by observing them in similar contexts across training examples. This implicit learning demands large amounts of training data and may fail to generalise to low-resource language pairs.
- **Symphonym (Explicit Features):** By using PanPhon articulatory features, I provide the model with vectors that already encode phonetic properties—both 'b' and 'p' are marked as `+bilabial`, `-continuant`, etc. The model need not learn these relationships; it is *informed* of them, enabling better generalisation to scripts and languages with limited training data.

This distinction is particularly important for toponym matching, where the long tail of historical languages and rare scripts cannot be adequately covered by implicit learning alone.

2.4 Cross-Lingual Representation Learning

The broader literature on cross-lingual embeddings provides relevant techniques. Conneau et al. (Conneau et al., 2018) show that monolingual embedding spaces can be aligned post-hoc using small bilingual dictionaries. Artetxe et al. (Artetxe et al., 2018) demonstrate unsupervised alignment using only monolingual corpora. However, these approaches target semantic similarity ("dog" near "chien") rather than phonetic similarity ("Paris" near "Париж").

The TRANSLIT resource (Benites et al., 2020) provides 1.6 million name variants across 180 languages for training transliteration systems. While valuable, such resources do not directly address the phonetic similarity task targeted here.

2.5 Phonetic Representations in NLP

The use of phonetic features in natural language processing has a long history. Epitran (Mortensen et al., 2018) provides rule-based grapheme-to-phoneme conversion for over 100 languages, outputting IPA transcriptions. PanPhon (Mortensen et al., 2016) maps IPA segments to articulatory feature vectors encoding phonological properties. These resources form the foundation of my Teacher network's phonetic grounding.

2.6 Siamese Networks for Similarity Learning

Siamese networks (Bromley et al., 1993; Chopra et al., 2005) learn similarity functions by processing two inputs through identical sub-networks and comparing their output representations. They have been successfully applied to face verification (Schroff et al., 2015), signature verification, and text similarity (Mueller and Thyagarajan, 2016; Neculoiu et al., 2016).

For text matching, Mueller and Thyagarajan (Mueller and Thyagarajan, 2016) demonstrated that Siamese LSTMs with Manhattan distance (L_1) achieve strong performance on sentence similarity tasks. However, the choice of distance metric carries important implications for toponym matching:

Manhattan Distance Limitations. The L_1 norm is additive across dimensions, which creates a *length penalty*: toponyms varying significantly in character count (e.g., "Oa" vs. "Constantinople") may produce large distances purely due to embedding magnitude rather than phonetic dissimilarity. Additionally, Manhattan distance assumes that the optimal path between embeddings follows axis-aligned grid lines, which poorly captures the directional relationships in phonetic space where the *ratio* of articulatory features often matters more than their absolute magnitudes.

Cosine Similarity Advantages. Cosine similarity normalises for vector magnitude, measuring only the angle between embeddings. For phonetic representations, this is preferable: two names with similar phonetic "direction" (ratio of nasality to voicing, place to manner, etc.) should be considered similar regardless of sequence length. I therefore use cosine similarity for inference and incorporate it into the training loss.

Lin et al. (Lin et al., 2017) introduced structured self-attention for sentence embeddings. My architecture extends this line of work by incorporating self-attention mechanisms and operating on phonetic feature sequences, with distance metrics chosen specifically for the phonetic domain.

2.7 Summary of Technical Distinctions

Table 1 summarises the key technical distinctions between my approach and related neural matching methods.

3 Architecture

My system employs a Teacher-Student architecture where a Teacher network, trained on articulatory phonetic features, produces target embeddings that a Student network learns to approximate from character sequences alone. At inference time, only the Student is required, enabling deployment without phonetic resources. This design addresses the limitations of prior approaches discussed in

Feature	Rama (2016)	Mueller & Thyagarajan	Sagi et al. (MEHDIE)	Symphonym
Input	Character embeddings	Word/sentence embeddings	Transliterated strings	Articulatory features (PanPhon)
Phonetic Knowledge	Implicit (learned)	None (semantic)	Rule-based translit.	Explicit (engineered)
G2P Method	None	None	Phonetisaurus	Epitran (100+ langs)
Distance Metric	Contrastive (Eucl.)	Manhattan (L_1)	Edit distance	Cosine + triplet
Scripts	Single	Single	Hebrew–Arabic	20 unified
Strength	Minimal preproc.	Gradient stability	Semitic affinity	Cross-lingual

Table 1: Comparison of neural matching approaches. Symphonym differs from prior work primarily in its use of explicit articulatory features and unified multi-script architecture.

Section 2, particularly the reliance on language-specific phonetic rules and single-script operation.

3.1 Design Principles

Three principles guide my architecture:

Script Awareness Without Script Dependence. The model must handle 20 writing systems but produce embeddings in a unified space where script boundaries are transparent. I achieve this through deterministic script detection (via Unicode block analysis) combined with learned script embeddings that allow the model to interpret characters appropriately for each script.

Phonetic Grounding. Embedding similarity must reflect phonetic similarity, not orthographic or semantic similarity. I ground the embedding space through the Teacher network, which operates on articulatory features derived from IPA transcriptions.

Inference Efficiency. The deployed model must encode toponyms without runtime phonetic conversion, language identification, or external resources. All phonetic knowledge is distilled into the Student's parameters during training.

3.2 Architecture Overview

Figure 1 illustrates the Teacher-Student architecture and the three-phase training curriculum.

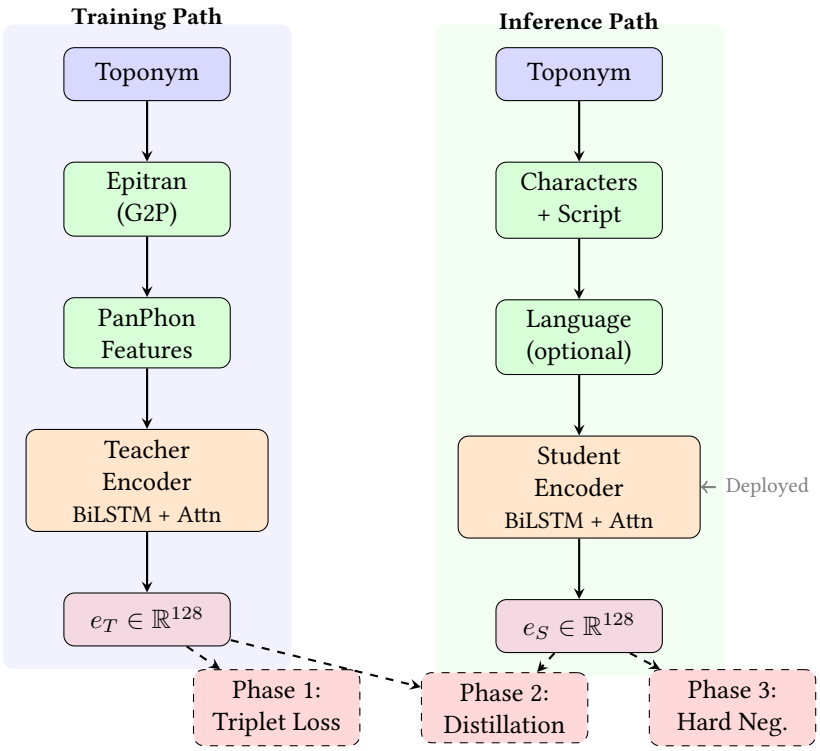

Figure 1: Teacher-Student architecture. **Left**: The Teacher pathway converts toponyms to IPA via Epitran, extracts PanPhon articulatory features, and encodes to 128-d embeddings. Used only during training. **Right**: The Student pathway processes raw characters with script/language metadata. Deployed at inference time without phonetic resources. **Bottom**: Three training phases progressively transfer phonetic knowledge from Teacher to Student.

3.3 Script Detection and Character Vocabulary

I detect script from Unicode code points, mapping each character to one of 20 script categories: Latin, Cyrillic, Greek, Arabic, Hebrew, Devanagari, Bengali, Tamil, Telugu, Malayalam, Kannada, Gujarati, Thai, Georgian, Armenian, Chinese (Han), Japanese (Hiragana, Katakana), Korean (Hangul), and Other.

Character vocabulary construction balances coverage against tractability:

- **Alphabetic scripts** (Latin, Cyrillic, Greek, Arabic, Hebrew, Brahmic scripts, Georgian, Armenian, Thai): I retain native characters, yielding vocabularies of 50–200 tokens per script.
- **Chinese and Japanese**: The tens of thousands of Han characters would create an intractable vocabulary. I romanise via AnyAscii, reducing to the Latin character set while preserving approximate phonetic content.

- **Korean Hangul**: Rather than treating 11,172 syllable blocks as atomic units, I decompose into Jamo (lead consonant, vowel, optional tail consonant), yielding approximately 60 tokens with explicit phonetic structure.

The resulting vocabulary contains approximately 11,000 tokens across all scripts, each tagged with its source script for embedding lookup.

3.4 Teacher Network: Phonetic Feature Encoder

The Teacher network encodes toponyms via their IPA transcriptions and articulatory features. Given a toponym, I first obtain an IPA transcription using Epitran (Mortensen et al., 2018), then extract PanPhon features (Mortensen et al., 2016)—a 24-dimensional binary vector per phoneme encoding articulatory properties (place, manner, voicing, etc.).

The Teacher architecture:

1. **Input**: Sequence of PanPhon feature vectors, $(f_1, f_2, \ldots, f_T)$ where $f_i \in \{0, 1\}^{24}$
2. **Projection**: Linear layer projects features to hidden dimension: $h_i = W_f f_i + b_f$
3. **BiLSTM**: Bidirectional LSTM processes the sequence: $(\overrightarrow{h_i}, \overleftarrow{h_i}) = \text{BiLSTM}(h_1, \ldots, h_T)$
4. **Self-Attention**: Multi-head self-attention captures long-range dependencies
5. **Attention Pooling**: Learned attention weights aggregate sequence to fixed-length vector
6. **Output Projection**: Linear layer to 128 dimensions with L2 normalisation

The Teacher is trained on triplets of co-located toponyms, learning to place phonetically similar names (same place, different languages) closer than phonetically dissimilar names (different places).

3.5 Generalisation Through Articulatory Features

A key advantage of grounding the embedding space in PanPhon articulatory features is *zero-shot generalisation* to languages and scripts not explicitly represented in training data. This property emerges from the universal nature of articulatory phonetics.

Language-Agnostic Feature Space. PanPhon represents each IPA segment as a 24-dimensional binary vector encoding articulatory properties that are universal across human languages: place of articulation (bilabial, alveolar, velar, etc.), manner of articulation (stop, fricative, nasal, etc.), voicing, and secondary features like aspiration and nasalisation. These features describe *how sounds are physically produced*, not which language they belong to. Consequently, the phoneme /b/ receives identical features whether it appears in English "Berlin", Russian "Берлин", or Arabic "برلين"—all three encode a voiced bilabial stop.

Implicit Cross-Lingual Transfer. When the Teacher network learns that two toponyms with similar PanPhon feature sequences should have similar embeddings, it learns relationships between articulatory configurations, not between specific languages. If the model learns during training that the Latin sequence "ber-lin" (voiced bilabial stop + mid front vowel + alveolar liquid + ...) should be close to the Cyrillic "бер-лин" (same articulatory sequence in different orthography), this knowledge transfers automatically to any other script that represents the same sounds—even scripts entirely absent from training.

Example: Unseen Script Generalisation. Consider Georgian, a script with limited representation in typical training corpora. If the model has learned from Latin–Cyrillic pairs that voiced bilabial stops followed by mid front vowels produce similar embeddings regardless of orthography, then Georgian "ბერლინი" (berlini) will naturally cluster with its Latin and Cyrillic equivalents, because its IPA transcription yields the same PanPhon features. The Student network, having learned to approximate Teacher embeddings, inherits this generalisation capability even though it operates on raw characters.

Phonological Universals as Inductive Bias. The articulatory feature representation encodes known phonological universals. Sounds that are perceptually similar across languages (e.g., /p/ and /b/ differing only in voicing) receive similar feature vectors, providing an inductive bias that helps the model learn meaningful phonetic relationships even from limited data. This contrasts with purely character-based approaches, which must learn from scratch that 'p' and 'b' are related—a relationship that may not be evident from co-occurrence statistics alone.

Teacher Model for Untrained Scripts. For scripts or languages where the Student network may produce suboptimal embeddings due to limited training coverage, the Teacher network can be used directly at inference time if IPA transcription is available. This is particularly valuable for scholarly applications involving historical or rare scripts: a researcher working with Old Church Slavonic or Ge'ez can obtain IPA transcriptions through specialist resources and use the

Teacher pathway to generate embeddings that participate fully in the phonetically-grounded embedding space, bypassing any limitations of the Student's character-level learning for these scripts.

3.6 Student Network: Universal Character Encoder

The Student network processes raw character sequences, optionally augmented with script and language information:

1. **Character Embedding**: Each character maps to a learned 96-dimensional vector via script-partitioned vocabulary lookup
2. **Script Embedding**: Deterministically detected script maps to a learned 16-dimensional vector, concatenated to each character embedding
3. **Language Embedding**: When available, ISO 639 language code maps to a learned 16-dimensional vector; otherwise, a special `<UNK>` token is used. During training, known languages are randomly replaced with `<UNK>` with 50% probability, forcing the model to learn script-intrinsic patterns.
4. **BiLSTM + Self-Attention + Attention Pooling**: Identical architecture to Teacher
5. **Output Projection**: Linear layer to 128 dimensions with L2 normalisation

3.7 Noise Augmentation

To handle OCR errors, historical spelling variations, and transcription inconsistencies, I apply character-level noise during Student training:

- **Insertion**: Random character from same script inserted (probability 0.1)
- **Deletion**: Random character deleted (probability 0.1)
- **Substitution**: Random character replaced with another from same script (probability 0.05)
- **Transposition**: Adjacent characters swapped (probability 0.05)

Noise is applied with 30% probability per training example, with the target being the Teacher's embedding of the *clean* input. This trains the Student to map corrupted inputs to the same phonetic region as their clean counterparts.

4 Training Data

4.1 Data Selection and Curation Criteria

The World Historical Gazetteer (WHG) infrastructure separates data into a `places` index, which aggregates geographic entities with their attributions, and a `toponyms` index, which stores individual name variants with script and language metadata. My data selection strategy leverages this architecture to filter high-quality training data from major namespace authorities—GeoNames (`gn`), Wikidata (`wd`), and the Getty Thesaurus of Geographic Names (`tgn`)—while excluding less strictly curated sources like OpenStreetMap.

Training requires pairs of toponyms that refer to the same place, which are used to construct the triplets described in Section 3. I extract these from co-located name attestations: if an authority lists "London", "Londres", "Londra", and "Лондон" within the same place record, I generate pairs among all combinations. To address the inherent script and language imbalances in the raw corpus, I define the training data selection based on six criteria.

1. Script-Stratified Sampling. Rather than ingesting all available pairs, which would result in a corpus heavily skewed toward Latin script and European languages, I implement a quota-based sampling strategy. I sample up to $N = 100,000$ pairs for each script-pair combination (e.g., Latin–Cyrillic, Latin–Devanagari, Arabic–Hebrew). This prevents high-resource scripts from dominating the gradient signal while ensuring sufficient coverage for low-resource script pairs.

2. Namespace Filtering. I restrict training data to records from the `gn`, `tgn`, and `wd` namespaces. The inclusion of Wikidata is critical for meeting the stratification quotas: Wikidata provides the majority of non-Latin script coverage, particularly for South Asian scripts (Devanagari, Tamil, Telugu, Kannada, Malayalam, Bengali) and Middle Eastern scripts (Arabic, Hebrew), where GeoNames and TGN lack sufficient multi-script correspondence.

3. Global Vocabulary Construction. To prevent out-of-vocabulary (OOV) errors at inference time, I decouple vocabulary construction from pair generation.

- **Pass 1 (Vocabulary):** I scan the entire corpus (112 million toponyms) to build the character vocabulary, applying script-specific preprocessing: CJK and Japanese Kana are romanised via AnyAscii, while Korean Hangul is decomposed into Jamo. The vocabulary is then expanded to include full Unicode ranges for each observed script, yielding 11,122 tokens.
- **Pass 2 (Training Pairs):** I generate candidate pairs from co-located toponyms, restricted to the stratified quotas defined above.

4. Orthographic Twin Retention. I retain "same-string, different-language" pairs (e.g., "Paris"@en vs "Paris"@fr) regardless of whether their pronunciation differs. The Teacher network, operating on IPA-derived articulatory features, provides ground truth: if pronunciation differs (e.g., "Paris" in French vs. English), the Teacher pushes embeddings apart; if pronunciation is stable (e.g., "Berlin"), it keeps them close. The Student receives identical character sequences but distinct language embeddings, learning when language context modifies phonetic interpretation.

5. Cross-Script Prioritisation. I weight cross-script pairs (e.g., 北京 vs. Beijing) higher than same-script variants (Beijing vs. Peking) during sampling. When approaching quota limits, cross-script pairs are 3× more likely to be accepted than same-script pairs, as bridging disjoint character spaces is the primary architectural goal.

6. Deduplication. Duplicate pairs arise from two sources: (a) the same place appearing in multiple authorities (e.g., London exists in GeoNames, Wikidata, and TGN, each providing the pair London@en/Londres@fr), and (b) different places sharing the same name variants (e.g., "Santiago" refers to cities in Chile, Spain, Cuba, and the Philippines, each generating Santiago@es/Santiago@en pairs). Without deduplication, identical toponym pairs would be massively over-represented. I maintain a global set of seen pairs and emit each unique (anchor, positive) combination only once, regardless of provenance.

4.2 IPA Transcription and Feature Extraction

The Teacher network requires IPA transcriptions. I use Epitran (Mortensen et al., 2018) for grapheme-to-phoneme conversion, which supports approximately 100 languages covering the majority of GeoNames entries. Epitran provides rule-based G2P that outputs IPA transcriptions suitable for PanPhon feature extraction. Names in languages without Epitran support are excluded from Teacher training but can still be processed by the Student, which learns to generalise from related scripts.

4.3 Phonetic Similarity Filtering

A critical challenge is **false cognates**: exonyms that refer to the same place but share no phonetic relationship. For example, "Germany" and "Deutschland", or "日本" (Nihon) and "Japan", denote the same entity but should not be trained as phonetically similar. Naively including such pairs would corrupt the embedding space.

I apply phonetic similarity filtering using a simple heuristic: after romanisation via AnyAscii, I compute normalised Levenshtein similarity between name

pairs. Pairs below a threshold of 0.35 are excluded from positive training examples. This successfully filters:

- "Germany" / "Deutschland" (similarity 0.18)
- "Japan" / "日本" → "Ri Ben" (similarity 0.22)
- "Finland" / "Suomi" (similarity 0.15)

While retaining genuine phonetic cognates:

- "London" / "Londres" (similarity 0.57)
- "München" / "Munich" (similarity 0.57)
- "Москва" → "Moskva" / "Moscow" (similarity 0.71)

4.4 Dataset Statistics

The full WHG `places` index contains 47.1 million place records, from which I extract 112.0 million toponym records spanning 1,944 languages and 20 script categories. During extraction, 1.77 million toponyms (1.6%) are filtered as "pre-romanised" forms—cases where the language tag implies a non-Latin script but the name is written in Latin characters (e.g., Chinese names romanised to Pinyin without explicit tagging). These are excluded to prevent the model from learning spurious orthographic similarities.

For training, I restrict to the three high-quality namespace authorities: GeoNames (`gn`), Wikidata (`wd`), and the Getty Thesaurus of Geographic Names (`tgn`). This yields 57.6 million training toponyms with the following script distribution:

The character vocabulary is constructed by scanning all 112M toponyms (not just training data), ensuring coverage of rare characters that may appear at inference time. After preprocessing—romanisation of CJK via AnyAscii and decomposition of Hangul to Jamo—the final vocabulary contains 11,122 tokens. The vocabulary is expanded to include full Unicode ranges for each observed script, ensuring the model can handle characters not seen in the training corpus.

Namespace Contribution. Within the training data, Wikidata contributes the majority of non-Latin script coverage. For example, in the Devanagari script, Wikidata provides 130,110 toponyms compared to GeoNames' 17,676 and TGN's 2,035. This pattern holds across most South Asian and Middle Eastern scripts, validating my decision to include Wikidata despite its heterogeneous quality.

Filtered Lang-Script Mismatches. The top filtered combinations were: Chinese/Latin (923,079), Persian/Latin (197,099), Japanese/Latin (111,945), Russian/Latin (100,805), and Kazakh/Latin (77,768). These represent pre-romanised forms that would otherwise introduce noise into the training signal.

Script	Count	%	Languages	Top Languages
LATIN	50,068,496	86.9%	874	en, ceb, fr, nl, de
CYRILLIC	2,903,698	5.0%	195	ru, uk, ce, tt, bg
CJK	1,857,832	3.2%	86	zh, ja, wuu, gan, yue
ARABIC	1,751,075	3.0%	129	fa, ar, arz, azb, ur
KATAKANA	310,691	0.5%	32	ja
HANGUL	270,407	0.5%	36	ko
OTHER	243,259	0.4%	401	lauc, my, en, pa, si
THAI	211,956	0.4%	32	th
GREEK	172,879	0.3%	72	el, grc, pnt
ARMENIAN	148,266	0.3%	39	hy, hyw
HEBREW	133,232	0.2%	49	he, yi
DEVANAGARI	131,785	0.2%	58	hi, mr, ne, sa
BENGALI	97,931	0.2%	28	bn, as
GEORGIAN	93,334	0.2%	33	ka
MALAYALAM	53,570	0.1%	8	ml
TAMIL	47,748	0.1%	14	ta
HIRAGANA	47,695	0.1%	18	ja
TELUGU	47,646	0.1%	14	te
KANNADA	21,372	0.04%	14	kn
GUJARATI	20,340	0.04%	6	gu

Table 2: Script distribution in training data (gn/wd/tgn namespaces). The "Languages" column shows the number of distinct ISO 639 language codes observed for each script.

Training Pair Generation. Positive pairs are generated from co-located toponyms within each place record. A place entry for "Paris" in Wikidata, for instance, contains variants in dozens of languages: Paris (en/fr), París (es), Париж (ru), باريس (ar), 巴黎 (zh), パリ (ja), etc. I generate pairs from all combinations of co-located toponyms that pass phonetic similarity filtering (normalised Levenshtein ≥ 0.35 for cross-script pairs, ≥ 0.60 for same-script pairs after romanisation). Same-script cross-language pairs (e.g., London@en vs Londres@fr) are sampled at 20% to prevent domination by orthographic twins.

From an estimated 428,956,662 candidate pairs across 66,924,548 toponyms, my pipeline scanned 174,636,677 before script-pair quotas (100K per combination) were exhausted. Of these, 130,329,534 (74.6%) passed similarity thresholds. After quota-based stratified sampling and deduplication (802,283 duplicates removed), the final training set contains **5,088,419 unique pairs**. While smaller than the raw count suggests, this curated set ensures balanced representation across script pairs and prevents any single script combination from dominating training. Each pair represents a genuine phonetic correspondence verified through co-location in authoritative gazetteer sources.

From these pairs, I generate triplets for contrastive training:

- **Phase 1 triplets:** 5,088,419 triplets with random negatives sampled from

the full toponym pool

- **Phase 3 triplets:** 5,088,419 triplets with hard negatives—orthographically similar names (same 2-character prefix, same script) from different places

The hard negative mining for Phase 3 uses a prefix index of 11,182,368 entries spanning 11,617 prefix/script combinations.

5 Training Methodology

I employ a three-phase curriculum that progressively builds from phonetic feature learning through knowledge distillation to discriminative fine-tuning.

5.1 Phase 1: Teacher Training on Phonetic Features

The Teacher network learns to produce embeddings where phonetically similar toponyms cluster together. Training uses triplet margin loss:

$$\mathcal{L}_{\text{triplet}} = \max(0, \|e_a - e_p\|_2 - \|e_a - e_n\|_2 + m) \tag{1}$$

where e_a, e_p, e_n are embeddings of anchor, positive (same place), and negative (different place) toponyms, and m is the margin (I use $m = 0.3$).

Negatives are sampled randomly from the full toponym pool for Phase 1, ensuring broad coverage of the phonetic space before hard negative mining in Phase 3.

Effective Training Set. Of the 5,088,419 generated triplets, only those where all three toponyms (anchor, positive, negative) have valid IPA transcriptions can be used for Teacher training. From 7,987,814 unique toponym IDs referenced in triplets, 3,663,966 (45.9%) have Epitran-supported language codes and valid PanPhon features. After filtering, **467,546 triplets** remain for training and **58,316** for validation. This reduction (to 9.2% of generated triplets) follows from the joint probability: if 45.9% of toponyms have features, the probability all three in a triplet have features is approximately $0.459^3 \approx 9.7\%$.

Despite this reduction, the training set remains adequate: 467K triplets $\times$ 50 epochs yields 23.4M training examples, comparable to successful Siamese network training regimes in prior work. The Student network in Phase 2, operating on raw characters rather than IPA, will generalise to the remaining languages and scripts not covered by Epitran.

Training Configuration.

- Model: PhoneticEncoder with **1,008,513 parameters**
- Optimiser: AdamW with weight decay 10^{-5}

- Learning rate: 10^{-3} with cosine annealing
- Batch size: 128 triplets
- Epochs: 50
- Training time: $\sim$3 minutes/epoch ($\sim$20 batches/second on A100)

Convergence. Phase 1 training converged smoothly, with training loss decreasing from 0.0228 (epoch 1) to 0.0002 (epoch 50) and validation loss from 0.0117 to **0.0040**. The best model was saved at epoch 48. The near-zero training loss indicates the Teacher successfully learned to separate co-located toponyms from random negatives in the phonetic feature space.

5.2 Phase 2: Student-Teacher Alignment

The Student learns to approximate Teacher embeddings from character sequences. Given a toponym with both character sequence (Student input) and PanPhon feature sequence (Teacher input), I minimise the distance between their embeddings:

$$\mathcal{L}_{\text{distill}} = \alpha \cdot \text{MSE}(e_S, e_T) + (1 - \alpha) \cdot (1 - \cos(e_S, e_T)) \tag{2}$$

where e_S and e_T are Student and Teacher embeddings, and $\alpha = 0.5$ balances MSE and cosine components.

During this phase:

- Language dropout (50%) forces learning of script-intrinsic patterns
- Noise augmentation (30%) trains robustness to input corruption
- Teacher weights are frozen

Effective Training Set. Phase 2 trains on individual toponyms (not triplets), requiring only that each sample has valid IPA features for the Teacher to generate target embeddings. This yields **23,223,184 training samples** and **2,902,533 validation samples**—substantially more than Phase 1's triplet-based training.

Training Configuration.

- Model: UniversalEncoder (Student) with **1,761,217 parameters**
- Vocabularies: 11,122 characters, 20 scripts, 1,944 languages
- Optimiser: AdamW with weight decay 10^{-5}
- Learning rate: 10^{-3} with cosine annealing
- Batch size: 128

- Epochs: 5 (early stopping; validation loss plateaued)
- Training time: ∼82 minutes/epoch (∼37 batches/second on A100)

Convergence. Phase 2 converged rapidly over 5 epochs:

Epoch	Train Loss	Val Loss	Val Similarity
1	0.0837	0.0381	0.9625
2	0.0766	0.0359	0.9647
3	0.0750	0.0347	0.9658
4	0.0742	0.0344	0.9661
5	0.0736	0.0342	0.9663

The final cosine similarity of **0.9663** indicates that Student embeddings align very closely with Teacher embeddings, successfully transferring the phonetic knowledge from IPA-based to character-based representations. Training was stopped after epoch 5 as validation loss improvements had diminished to $<1\%$ per epoch.

5.3 Phase 3: Discriminative Fine-Tuning

While the distillation phase aligns the Student with the Teacher's general phonetic space, the Student may still struggle with *minimal pairs*—names with high orthographic similarity but distinct phonetic realizations (e.g., "Austria" vs. "Australia", or "Laughter" vs. "Slaughter").

In this final phase, I fine-tune the Student using triplet loss to sharpen these boundaries without degrading the learned cross-script equivalences.

Hard Negative Mining Strategy. I define negatives by mining triplets (a, p, n) where:

- **Anchor (a) & Positive (p):** A pair of toponyms confirmed to refer to the same place through co-location in authoritative gazetteer records.
- **Hard Negative (n):** A toponym selected such that it has high orthographic similarity to the anchor (same 2-character prefix after romanisation, same script) but is not known to refer to the same place.

The "different place" constraint is enforced through an **adjacency set** containing all 10.2 million toponym pairs that co-occur within place records. A candidate negative is rejected if it appears in any positive pair with the anchor.

This approach has a known limitation: it cannot detect cross-authority duplicates. If "Springfield" appears in both GeoNames and Wikidata records for the same city but those records are not linked in my index, the Wikidata toponym could be incorrectly selected as a hard negative for the GeoNames toponym. I mitigate this by (a) the relatively low probability of such collisions given 67M

toponyms and prefix-based sampling, and (b) the observation that even when such "false negatives" occur, they typically have identical or near-identical embeddings, producing negligible gradient signal. The triplet loss margin prevents the model from over-optimising on pairs that are already well-separated.

Crucially, I explicitly **exclude** co-located toponyms from being selected as negatives. If two toponyms share a place record (e.g., "Munich" and "München" in the same Wikidata entry), they are excluded via the adjacency check regardless of their surface similarity.

Effective Training Set. Phase 3 loads 5,088,419 hard negative triplets. After filtering for valid toponyms (those with character encodings within the vocabulary), **3,333,767 triplets** remain for training and **417,335** for validation. The higher retention rate (65.5%) compared to Phase 1 (9.2%) reflects that Phase 3 operates on the character-based Student, which does not require Epitran/IPA coverage.

Training Configuration.

- **Loss Function:** Triplet Margin Loss ($m = 0.2$).
- **Teacher Status:** Frozen (used only to validate true phonetic distances for negative sampling).
- **Optimiser:** AdamW with a reduced learning rate (1×10^{-5}) to prevent catastrophic forgetting of the cross-script alignment learned in Phase 2.
- **Epochs:** 30
- **Training time:** $\sim$22 minutes/epoch ($\sim$20 batches/second on A100)

Trade-off: Cross-Script vs. Same-Script Performance. Hard negative training sharpens discrimination between orthographically similar names at a modest cost to same-script cross-language pairs. Between epochs 5 and 6, cross-script accuracy improved from 28/29 to 29/29 (Thessaloniki/$\Theta\varepsilon\sigma\sigma\alpha\lambda ονίκη$ rose from 0.848 to 0.929), while same-script pairs like London/Londra declined slightly (0.617 to 0.598). This is expected: hard negatives are sampled from the *same* script, teaching the model to be more discriminating within scripts. Since Symphonym's primary goal is cross-script matching—linking "北京" to "Beijing" rather than distinguishing "London" from "Londres"—this trade-off is acceptable. Same-script variants remain linked at the place level through co-location in gazetteer records. Moreover, same-script matching can be effectively handled by traditional string similarity methods (Levenshtein distance, Jaro-Winkler) that complement Symphonym's cross-script capabilities; a hybrid pipeline might apply Symphonym for cross-script candidate retrieval and fall back to edit-distance methods for same-script refinement.

5.4 Implementation Details

Training uses PyTorch on NVIDIA A100 GPUs. The Teacher contains **1,008,513 parameters**; the Student contains **1,761,217 parameters** (larger due to the 11,122-token character vocabulary). Phase 1 completes in approximately 2.5 hours (50 epochs $\times$ 3 minutes/epoch); Phase 2 completes in approximately 7 hours (5 epochs $\times$ 82 minutes/epoch); Phase 3 requires approximately 11 hours (30 epochs $\times$ 22 minutes/epoch).

The embedding dimension of 128 was chosen to balance expressiveness against storage and retrieval costs. Elasticsearch's HNSW index stores embeddings directly, so smaller dimensions reduce index size and improve cache efficiency. Experiments with 64, 128, and 256 dimensions showed diminishing returns above 128 for toponym matching.

Checkpoints are saved after each epoch, with the best model selected by validation loss. Training is deterministic given fixed random seeds, enabling reproducibility. I use mixed-precision training (FP16) for efficiency without measurable accuracy loss.

5.5 Training Curves

[Figure ?? will show training and validation loss for all three phases once training completes.]

6 Evaluation

6.1 Preliminary Results

[DEPRECATED: THESE RESULTS ARE FROM A PROTOTYPE MODEL]

I present interim results from initial training runs to characterise model behaviour prior to full evaluation. These results use the Phase 2 checkpoint (Student-Teacher alignment complete, before hard negative fine-tuning).

Cross-script equivalents. As shown in Table 3, these pairs achieve high similarity as intended.

Name 1	Name 2	Similarity
Moskva	Москва	0.998
Parizh	Париж	0.995
Moscow	Moscou	0.986
London	Лондон	0.997

Table 3: Cross-script transliteration pairs.

Historical variants. Diachronic name changes (Paris–Lutetia, Istanbul–Constantinople) yield moderate to low similarity, as expected. The model captures *synchronic* phonetic similarity, not historical derivation. Linking ancient to modern names is handled at the place level through curated attestation records. Table 4 shows representative pairs.

Modern	Historical	Similarity
London	Londinium	0.938
Cologne	Colonia	0.968
Lyon	Lugdunum	0.683
Istanbul	Constantinople	0.582

Table 4: Historical variant pairs. Lower scores for phonetically distant pairs (Lyon–Lugdunum, Istanbul–Constantinople) are expected and correct; historical linkage is handled through place-level attestation records rather than phonetic inference.

Unrelated pairs. Conversely, unrelated pairs are appropriately separated (Table 5).

Name 1	Name 2	Similarity
London	Tokyo	0.335
Paris	Beijing	0.268
Berlin	Cairo	−0.116

Table 5: Unrelated toponym pairs.

The pairwise similarity distribution across 22 diverse toponyms shows mean 0.092 ($\sigma = 0.348$), indicating the embedding space has not collapsed and maintains discrimination between unrelated names.

Limitations observed. Same-script cross-lingual variants show weaker performance than cross-script pairs: London–Londres (0.754) and Munich–München (0.923) fall below the 0.95 target, suggesting the model relies partly on script differences as a cue for phonetic equivalence. Phase 3 hard negative training aims to address this but requires careful curation to avoid introducing false equivalences.

6.2 Evaluation Tasks

[This section describes the evaluation methodology. Results are pending completion of Phase 3 training.]

I evaluate on four tasks designed to test the core claims of the system described in Section 5:

Task 1: Cross-Script Retrieval. Given a toponym in one script, retrieve its phonetic equivalents in other scripts. I sample places from Wikidata that have labels (`skos:altLabel`) in at least 3 distinct scripts. For each place, I use one script variant as query and measure retrieval of the other script variants. Importantly, Wikidata was not used in training, providing an independent test set.

Metrics: Recall@1, Recall@5, Recall@10, Mean Reciprocal Rank (MRR).

Task 2: Noise Robustness. Measure embedding stability under synthetic noise simulating OCR errors and transcription mistakes. For each test toponym, I generate corrupted variants at increasing noise levels (5%, 10%, 20% character error rate) and measure cosine similarity between clean and corrupted embeddings.

Metrics: Mean cosine similarity at each noise level; retrieval accuracy for corrupted queries.

Task 3: Historical Variant Linking. Link historical spelling variants to their attested modern or period-appropriate forms. I use two approaches:

- Index Villaris 17th-century spellings linked to place names from Ordnance Survey maps c.1900 (the GB1900 project) and, where available, to Wikidata entries. Since Index Villaris was not used in training, this provides an independent historical test set.
- Pleiades classical toponyms linked to their modern equivalents via Wikidata. Pleiades was similarly excluded from training, enabling unbiased evaluation on ancient-to-modern name correspondences.

A key methodological strength is that training on GeoNames, Wikidata, and TGN while evaluating on Pleiades and Index Villaris avoids train-test contamination for historical variant tasks.

Metrics: Linking accuracy (correct form in top-k).

Task 4: Scalability. Measure retrieval performance over the full WHG corpus of 67M toponym records using Elasticsearch's approximate k-NN with HNSW indexing.

Metrics: Query latency (p50, p95, p99); recall at various index configurations.

6.3 Baselines

I compare against:

- **Levenshtein on AnyAscii**: Edit distance after romanisation

- **Jaro-Winkler on AnyAscii**: String similarity after romanisation
- **Double Metaphone**: Phonetic codes after romanisation
- **Elasticsearch fuzzy match**: Built-in fuzzy query with fuzziness=AUTO
- **mBERT cosine similarity**: Multilingual BERT [CLS] embeddings

MEHDIE Benchmark. I additionally evaluate on the MEHDIE Hebrew-Arabic toponym benchmark (Sagi et al., 2025), which provides annotated place matches from medieval geographical texts. This benchmark tests cross-script matching between Hebrew and Arabic—a challenging case where phonetic approaches should excel compared to romanisation-based methods.

6.4 Results

MEHDIE Benchmark Results. Table 6 presents retrieval performance on the MEHDIE benchmark using ranking-based metrics appropriate for embedding systems. Symphonym significantly outperforms traditional string similarity baselines.

Method	R@1	R@5	R@10	MRR
Levenshtein (romanised)	0.815	0.976	0.994	0.885
Jaro-Winkler (romanised)	0.785	0.963	0.978	0.863
Symphonym	**0.892**	**0.988**	0.988	**0.930**

Table 6: MEHDIE benchmark results (average across 5 testsets, 137 ground truth matches). R@k = Recall at rank k; MRR = Mean Reciprocal Rank. Higher is better. Symphonym achieves 89.2% Recall@1, meaning the correct cross-script match is ranked first in nearly 9 out of 10 queries.

Symphonym achieves perfect Recall@1 (1.000) on testset 8 (Kima Al-Sham ↔ Thurayya Al-Sham) and 94.4% on testset 9 (Tudela ↔ Thurayya). The improvement over baselines is most pronounced on testset 10 (Yaqut ↔ Kima), where Symphonym achieves 84.8% R@1 compared to 66.7% for Levenshtein and 54.5% for Jaro-Winkler.

Comparison with MEHDIE System. Direct comparison with the MEHDIE system (Sagi et al., 2025) requires care because the systems optimise for different use cases. MEHDIE reports F-5 scores (recall weighted 5× over precision) at fixed similarity thresholds, reflecting a *classification* task: given a threshold, which pairs should be flagged as matches? Their phonetic method achieves F-5 scores of 0.68–0.88 across testsets at $\theta = 0.9$.

Symphonym is designed for *retrieval*: given a query, rank all candidates by similarity and return the best matches. This reflects typical gazetteer reconciliation workflows where users review a ranked list rather than accept/reject based on a threshold. For this task, ranking metrics (Recall@k, MRR) are more appropriate.

The approaches are complementary:

- **MEHDIE** uses IPA transcription via Phonetisaurus G2P, then computes Hamming distance over phonetic feature vectors. This requires language-specific G2P models and explicit phonetic conversion at query time.
- **Symphonym** learns to map characters directly to a phonetic embedding space, requiring no runtime phonetic conversion. The model generalises to script pairs not seen in training (e.g., Hebrew-Arabic).

Both systems outperform simple romanisation-based baselines, confirming that phonetic similarity is valuable for cross-script toponym matching. Symphonym's advantage is operational: it requires only character input and scales to real-time search over millions of toponyms via approximate nearest neighbour indexing.

These results validate the core design: Symphonym correctly ranks phonetically equivalent cross-script names above unrelated alternatives, even for script pairs (Hebrew-Arabic) not explicitly represented in training data.

[Additional evaluation results pending (cross-script retrieval, noise robustness, historical variants).]

7 Discussion

[Discussion pending results.]

7.1 Limitations

Several limitations warrant acknowledgment:

Training Data Bias. Despite the stratified sampling strategy described in Section 4.1, my training sources retain biases. GeoNames over-represents populated places with official names, potentially under-representing historical variants. Wikidata's coverage depends on volunteer contributions, which skew toward places of encyclopaedic interest. TGN emphasises art-historically significant locations. Performance on under-represented scripts (e.g., Ethiopic, Myanmar, Tibetan) and on mundane places lacking multi-lingual attestations may be weaker.

Tonal Languages. The current architecture does not explicitly model tone, which is phonemically contrastive in Chinese, Vietnamese, Thai, and other languages. The PanPhon features described in Section 3.5 do not encode suprasegmental properties. Tonal minimal pairs (e.g., Mandarin place names differing only in tone) may not be distinguished.

IPA Coverage. The Teacher's phonetic grounding depends on Epitran's G2P coverage. Languages without Epitran support contribute only through the Student's character-level learning, which may learn sub-optimal representations for these scripts.

Computational Cost. Embedding 60M toponyms requires significant compute. While inference is fast (thousands of embeddings per second on GPU), full corpus re-embedding after model updates takes several hours.

8 Conclusion

I have presented a neural embedding system for cross-script toponym matching that places phonetically similar names near each other in a unified 128-dimensional space, regardless of writing system. The Teacher-Student architecture enables training on articulatory phonetic features while requiring only character sequences at inference time. Phonetic similarity filtering prevents learning of false cognates, and noise-aware training with language dropout forces the model to learn script-intrinsic phonetic patterns.

[Summary of results and impact pending.]

The system is to be deployed in the World Historical Gazetteer, enabling fuzzy phonetic search across 60M+ toponyms from antiquity to the present. I release the trained models and training code to support further research in historical toponym linking.

Data Availability

The training data are derived from publicly available sources: GeoNames (https://www.geonames.org/), Wikidata (https://www.wikidata.org/), and the Getty Thesaurus of Geographic Names (https://www.getty.edu/research/tools/vocabularies/tgn/). The MEHDIE benchmark used for evaluation is available via the supplementary file link at https://link.springer.com/article/10.1007/s10579-025-09812-9. Extracted training datasets and pre-computed embeddings will be made available upon publication.

Code Availability

Training code, model weights, and inference utilities are available at https://github.com/WorldHistoricalGaze
The system integrates with Elasticsearch for scalable deployment.

Acknowledgments

This research was supported in part by the University of Pittsburgh Center for Research Computing and Data, RRID:SCR_022735, through the resources provided. Specifically, this work used the HTC cluster, which is supported by NIH award number S10OD028483, and the H2P cluster, which is supported by NSF award number OAC-2117681.

[Additional acknowledgments pending.]

References

Mikel Artetxe, Gorka Labaka, and Eneko Agirre. A robust self-learning method for fully unsupervised cross-lingual mappings of word embeddings. In Iryna Gurevych and Yusuke Miyao, editors, *Proceedings of the 56th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 789–798, Melbourne, Australia, July 2018. Association for Computational Linguistics. doi: 10.18653/v1/P18-1073. URL https://aclanthology.org/P18-1073/.

Fernando Benites, Gilbert François Duivesteijn, Pius von Däniken, and Mark Cieliebak. TRANSLIT: A large-scale name transliteration resource. In *Proceedings of the 12th Conference on Language Resources and Evaluation (LREC)*, pages 3265–3271, 2020. URL https://aclanthology.org/2020.lrec-1.399/.

Jane Bromley, Isabelle Guyon, Yann LeCun, Eduard Säckinger, and Roopak Shah. Signature verification using a "siamese" time delay neural network. In *Advances in Neural Information Processing Systems (NIPS)*, volume 6, pages 737–744, 1993. URL https://proceedings.neurips.cc/paper/1993/file/288cc0ff022877bd3df94bc9360b9c5d-Paper.pdf.

Sumit Chopra, Raia Hadsell, and Yann LeCun. Learning a similarity metric discriminatively, with application to face verification. In *Proceedings of the IEEE Computer Society Conference on Computer Vision and Pattern Recognition (CVPR)*, volume 1, pages 539–546, 2005. doi: 10.1109/CVPR.2005.202.

Alexis Conneau, Guillaume Lample, Marc'Aurelio Ranzato, Ludovic Denoyer, and Hervé Jégou. Word translation without parallel data. In *Proceedings of the International Conference on Learning Representations*, 2018.

Karl E. Grossner and Ruth Mostern. Linked places in World Historical Gazetteer. In *Proceedings of the 4th ACM SIGSPATIAL International Workshop on Geospatial Humanities (GeoHumanities)*, pages 40–43, 2021. doi: 10.1145/3486187.3490203.

Grzegorz Kondrak. A new algorithm for the alignment of phonetic sequences. In *Proceedings of the 1st North American Chapter of the Association for Computational Linguistics Conference*, NAACL 2000, page 288–295, USA, 2000. Association for Computational Linguistics.

Zhouhan Lin, Minwei Feng, Cicero Nogueira Santos, Mo Yu, Bing Xiang, Bowen Zhou, and Yoshua Bengio. A structured self-attentive sentence embedding. In *Proceedings of the International Conference on Learning Representations (ICLR)*, 2017. URL https://arxiv.org/abs/1703.03130.

David R. Mortensen, Patrick Littell, Akash Bharadwaj, Kartik Goyal, Chris Dyer, and Lori Levin. Panphon: A resource for mapping IPA segments to articulatory feature vectors. In *Proceedings of the 26th International Conference on Computational Linguistics (COLING)*, pages 3475–3484, 2016. URL https://aclanthology.org/C16-1328/.

David R. Mortensen, Siddharth Dalmia, and Patrick Littell. Epitran: Precision G2P for many languages. In *Proceedings of the Eleventh International Conference on Language Resources and Evaluation (LREC)*, pages 2634–2639, 2018. URL https://aclanthology.org/L18-1140/.

Jonas Mueller and Aditya Thyagarajan. Siamese recurrent architectures for learning sentence similarity. In *Proceedings of the Thirtieth AAAI Conference on Artificial Intelligence*, pages 2786–2792, 2016. URL https://dl.acm.org/doi/10.5555/3016100.3016291.

Paul Neculoiu, Maarten Versteegh, and Mihai Rotaru. Learning text similarity with siamese recurrent networks. In *Proceedings of the 1st Workshop on Representation Learning for NLP*, pages 148–157, 2016. URL https://aclanthology.org/W16-1617/.

Josef R. Novak, Nobuaki Minematsu, and Keikichi Hirose. Phonetisaurus: Exploring grapheme-to-phoneme conversion with joint n-gram models in the WFST framework. *Natural Language Engineering*, 22(6):907–938, 2016. doi: 10.1017/S1351324915000315.

Lawrence Philips. Hanging on the metaphone. *Computer Language*, 7(12):39–43, 1990.

Lawrence Philips. The double metaphone search algorithm. *C/C++ Users Journal*, 18(6):38–43, 2000. doi: 10.5555/349124.349132.

Qinjun Qiu, Haiyan Li, Xinxin Hu, Miao Tian, Kai Ma, and Yunqiang Zhu. A deep neural network model for chinese toponym matching with geographic pre-training model. *International Journal of Digital Earth*, 17(1):2353111, 2024. doi: 10.1080/17538947.2024.2353111. Proposal of the GeoBERT model.

Taraka Rama. Siamese convolutional networks for cognate identification. In *Proceedings of COLING 2016, the 26th International Conference on Computational Linguistics: Technical Papers*, pages 1018–1027, 2016. URL https://aclanthology.org/C16-1097/.

Gabriel Recchia and Max Louwerse. A comparison of string similarity measures for toponym matching. In *Vagueness in Communication*, pages 54–67. Springer, 2013. doi: 10.1007/978-3-642-18446-8_4.

Robert C. Russell. Index. U.S. Patent 1,261,167, 1918. URL https://patents.google.com/patent/US1261167A/. Patented Apr. 2, 1918.

Tomer Sagi, Moran Zaga, Sinai Rusinek, Marcell Richard Fekete, Johannes Bjerva, and Katja Hose. Utilizing phonetic similarity for cross-source and cross-language toponym matching: a benchmark and prototype. *Language Resources and Evaluation*, 59(3):2427–2451, 2025. doi: 10.1007/s10579-025-09812-9.

Rodrigo Santos, Patricia Murrieta-Flores, Pablo Calafiore, and Bruno Martins. Learning to combine multiple string similarity metrics for effective toponym matching. *International Journal of Digital Earth*, 11(9):949–965, 2018. doi: 10.1080/17538947.2017.1371253.

Florian Schroff, Dmitry Kalenichenko, and James Philbin. Facenet: A unified embedding for face recognition and clustering. *2015 IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 815–823, 2015. doi: 10.1109/CVPR.2015.7298682.